

```
!pip -q install speechbrain openai-whisper librosa soundfile scipy pandas matplotlib
```

```

      803.2/803.2 kB 9.6 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
      2.3/2.3 MB 55.1 MB/s eta 0:00:00
      248.0/248.0 MB 1.5 MB/s eta 0:00:00
      121.6/121.6 kB 8.1 MB/s eta 0:00:00
      782.1/782.1 kB 34.5 MB/s eta 0:00:00
Building wheel for openai-whisper (pyproject.toml) ... done

```

```

import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import librosa
import librosa.display
import soundfile as sf
import torch

from scipy.signal import welch

warnings.filterwarnings("ignore")

SR = 16000
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

print("Device:", DEVICE)
print("Sample rate:", SR)

```

```

Device: cpu
Sample rate: 16000

```

```

# =====
# CELL 3 - EDGE-AI HUMAN VOICE DETECTION (VAD)
# =====

import numpy as np
import librosa
import matplotlib.pyplot as plt

print("Running Human Voice Detection...")

# -----
# Parameters
# -----

VAD_FRAME = 512
VAD_HOP = 256

# Human speech frequency range
LOW_FREQ = 300
HIGH_FREQ = 3400

# -----
# Create STFT
# -----

D = librosa.stft(
    audio,
    n_fft=1024,
    hop_length=VAD_HOP,
    win_length=1024
)

magnitude = np.abs(D)

freqs = librosa.fft_frequencies(
    sr=SR,
    n_fft=1024
)

# -----

```

```

# Speech frequency mask
# -----

speech_band = (
    (freqs >= LOW_FREQ) &
    (freqs <= HIGH_FREQ)
)

# Energy in speech frequency band
speech_energy = np.mean(
    magnitude[speech_band, :] ** 2,
    axis=0
)

# Total energy
total_energy = np.mean(
    magnitude ** 2,
    axis=0
)

# -----
# Speech-band ratio
# -----

speech_ratio = (
    speech_energy /
    (total_energy + 1e-10)
)

# -----
# Normalize
# -----

speech_ratio_norm = (
    speech_ratio -
    np.min(speech_ratio)
) / (
    np.max(speech_ratio) -
    np.min(speech_ratio) +
    1e-10
)

# -----
# Voice probability
# -----

VOICE_THRESHOLD = 0.30

voice_probability = speech_ratio_norm

speech_detected = (
    voice_probability >= VOICE_THRESHOLD
)

# -----
# Convert frame decision to time
# -----

times = librosa.frames_to_time(
    np.arange(len(voice_probability)),
    sr=SR,
    hop_length=VAD_HOP
)

# -----
# Results
# -----

speech_percentage = (
    np.mean(speech_detected) * 100
)

print(
    f"Human Voice Detected in "
    f"{speech_percentage:.2f}% of audio"
)

```

```

if speech_percentage > 5:
    print("STATUS: HUMAN VOICE DETECTED")
else:
    print("STATUS: NO SIGNIFICANT HUMAN VOICE DETECTED")

# -----
# Plot Voice Probability
# -----

plt.figure(figsize=(14, 5))

plt.plot(
    times,
    voice_probability,
    label="Voice Probability"
)

plt.axhline(
    VOICE_THRESHOLD,
    linestyle="--",
    label="Voice Detection Threshold"
)

plt.xlabel("Time (seconds)")
plt.ylabel("Voice Probability")
plt.title("Edge-AI Human Voice Detection / VAD")
plt.ylim(0, 1)
plt.grid(True)
plt.legend()
plt.show()

```

```

from google.colab import files

uploaded = files.upload()

if not uploaded:
    raise RuntimeError("No audio file uploaded.")

INPUT_FILE = list(uploaded.keys())[0]

print("Input:", INPUT_FILE)

```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving mm vedios.mp3 to mm vedios.mp3

```

audio, _ = librosa.load(
    INPUT_FILE,
    sr=SR,
    mono=True
)

audio = audio.astype(np.float32)

if len(audio) == 0:
    raise RuntimeError("Audio is empty.")

print("Duration:", round(len(audio) / SR, 2), "seconds")
print("Samples:", len(audio))

```

Duration: 8.41 seconds
Samples: 134490

```

# Estimate noise from quietest 20% of frames

FRAME = 1024
HOP = 256

frames = librosa.util.frame(
    audio,
    frame_length=FRAME,
    hop_length=HOP
)

energy = np.mean(frames ** 2, axis=0)

```

```

threshold = np.percentile(energy, 20)

quiet = frames[:, energy <= threshold]

print("Total frames:", frames.shape[1])
print("Noise-estimation frames:", quiet.shape[1])

noise_psd_list = []

for i in range(quiet.shape[1]):
    f, p = welch(
        quiet[:, i],
        fs=SR,
        nperseg=FRAME
    )
    noise_psd_list.append(p)

if noise_psd_list:
    NOISE_PSD = np.mean(
        np.asarray(noise_psd_list),
        axis=0
    )
else:
    f, NOISE_PSD = welch(
        audio,
        fs=SR,
        nperseg=FRAME
    )

NOISE_PSD_DB = 10 * np.log10(
    NOISE_PSD + 1e-12
)

print("PSD noise profile ready.")

```

```

Total frames: 522
Noise-estimation frames: 105
PSD noise profile ready.

```

```

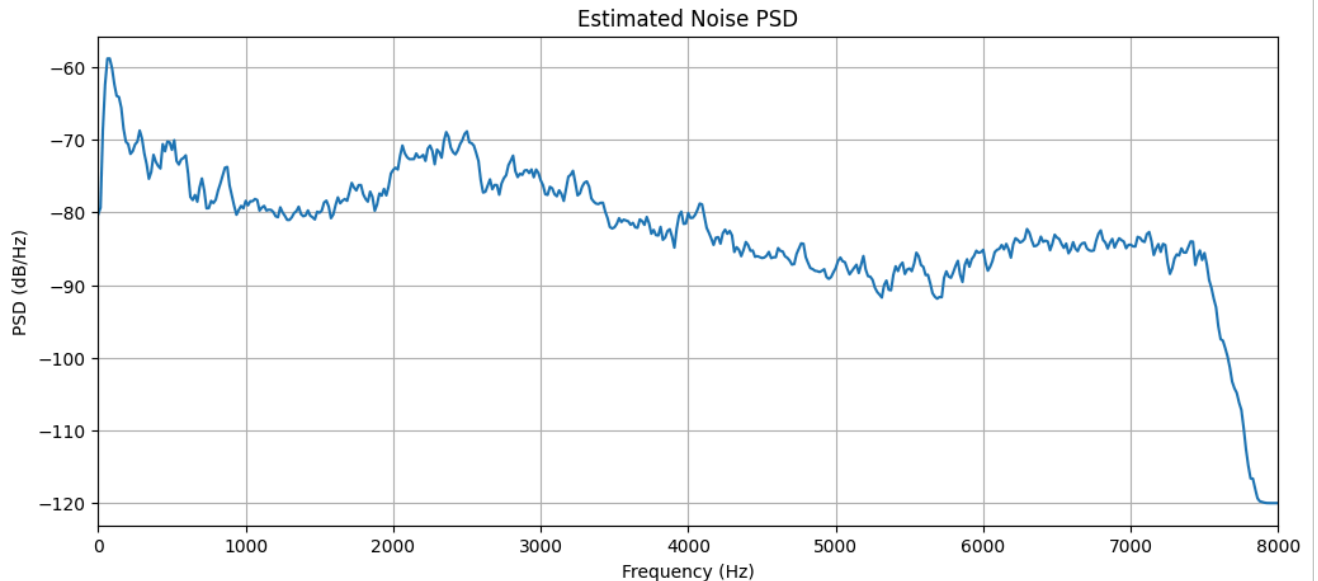
plt.figure(figsize=(12, 5))

plt.plot(
    f,
    NOISE_PSD_DB
)

plt.xlim(0, 8000)
plt.xlabel("Frequency (Hz)")
plt.ylabel("PSD (dB/Hz)")
plt.title("Estimated Noise PSD")
plt.grid(True)

plt.show()

```



```
from speechbrain.inference.separation import SepformerSeparation
```

```
print("Loading AI enhancement model...")
```

```
model = SepformerSeparation.from_hparams(
    source="speechbrain/sepformer-dns4-16k-enhancement",
    savedir="/content/sepformer_dns4",
    run_opts={"device": DEVICE}
)
```

```
print("Model loaded.")
```

```
INFO:speechbrain.utils.fetching:Fetch hyperparams.yaml: Fetching from HuggingFace Hub 'speechbrain/sepformer-dns4-16k-enhancement'
Loading AI enhancement model...
```

```
hyperparams.yaml: 100% 1.57k/1.57k [00:00<00:00, 97.2kB/s]
```

```
INFO:speechbrain.utils.fetching:Fetch encoder.ckpt: Fetching from HuggingFace Hub 'speechbrain/sepformer-dns4-16k-enhancement' i
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
```

```
encoder.ckpt: reconstructing file: 100% 17.3kB / 17.3kB, 1.70kB/s
```

```
encoder.ckpt: downloading bytes: 16.3kB, 1.60kB/s
```

```
INFO:speechbrain.utils.fetching:Fetch masknet.ckpt: Fetching from HuggingFace Hub 'speechbrain/sepformer-dns4-16k-enhancement' i
```

```
masknet.ckpt: reconstructing file: 100% 113MB / 113MB, 10.6MB/s
```

```
masknet.ckpt: downloading bytes: 99.6MB, 9.01MB/s
```

```
INFO:speechbrain.utils.fetching:Fetch decoder.ckpt: Fetching from HuggingFace Hub 'speechbrain/sepformer-dns4-16k-enhancement' i
```

```
decoder.ckpt: reconstructing file: 100% 17.2kB / 17.2kB, 1.70kB/s
```

```
decoder.ckpt: downloading bytes: 15.8kB, 1.56kB/s
```

```
INFO:speechbrain.utils.parameter_transfer:Loading pretrained files for: encoder, masknet, decoder
Model loaded.
```

```
print("Enhancing audio...")
```

```
result = model.separate_file(
    path=INPUT_FILE
)
```

```
print("Model output:", tuple(result.shape))
```

```
Enhancing audio...
Resampling the audio from 44100 Hz to 16000 Hz
Model output: (1, 134490, 1)
```

```
enhanced_audio = (
    result[:, :, 0]
    .detach()
```

```

        .cpu()
        .numpy()
    )

    enhanced_audio = np.squeeze(
        enhanced_audio
    ).astype(np.float32)

    enhanced_audio = enhanced_audio.reshape(-1)

    # Match original length without random cropping

    if len(enhanced_audio) > len(audio):
        enhanced_audio = enhanced_audio[:len(audio)]

    elif len(enhanced_audio) < len(audio):
        enhanced_audio = np.pad(
            enhanced_audio,
            (0, len(audio) - len(enhanced_audio))
        )

    print("Input samples :", len(audio))
    print("Output samples:", len(enhanced_audio))

```

```

Input samples : 134490
Output samples: 134490

```

```

EPS = 1e-10

input_rms = np.sqrt(
    np.mean(audio ** 2) + EPS
)

output_rms = np.sqrt(
    np.mean(enhanced_audio ** 2) + EPS
)

gain = input_rms / (output_rms + EPS)

# Avoid extreme volume changes
gain = np.clip(gain, 0.80, 1.25)

enhanced_audio = enhanced_audio * gain

# Safety limiter
peak = np.max(
    np.abs(enhanced_audio)
)

if peak > 0.98:
    enhanced_audio *= 0.98 / peak

print("Gain:", round(float(gain), 4))

```

```

Gain: 0.8

```

```

OUTPUT_FILE = "/content/enhanced_audio.wav"

sf.write(
    OUTPUT_FILE,
    enhanced_audio,
    SR
)

print("Saved:", OUTPUT_FILE)

```

```

Saved: /content/enhanced_audio.wav

```

```

from IPython.display import Audio, display

print("BEFORE")
display(Audio(audio, rate=SR))

print("AFTER")
display(Audio(enhanced_audio, rate=SR))

```

BEFORE

0:00 / 0:08

AFTER

0:00 / 0:08

```
def rms_db(x):
    return float(
        20 * np.log10(
            np.sqrt(np.mean(x**2) + EPS)
        )
    )
```

```
def peak_db(x):
    return float(
        20 * np.log10(
            np.max(np.abs(x)) + EPS
        )
    )
```

```
def noise_floor(x):
    r = librosa.feature.rms(
        y=x,
        frame_length=1024,
        hop_length=256
    )[0]

    return float(
        np.percentile(
            20 * np.log10(r + EPS),
            20
        )
    )
```

```
def speech_energy(x):
    D = librosa.stft(
        x,
        n_fft=1024,
        hop_length=256
    )

    power = np.abs(D)**2

    freq = librosa.fft_frequencies(
        sr=SR,
        n_fft=1024
    )

    mask = (
        (freq >= 300) &
        (freq <= 3400)
    )

    return float(
        10 * np.log10(
            np.mean(power[mask]) + EPS
        )
    )
```

```
def high_energy(x):
    D = librosa.stft(
        x,
        n_fft=1024,
        hop_length=256
    )

    power = np.abs(D)**2

    freq = librosa.fft_frequencies(
```

```

        sr=SR,
        n_fft=1024
    )

    mask = freq >= 5000

    return float(
        10 * np.log10(
            np.mean(power[mask]) + EPS
        )
    )

def estimated_snr(x):
    r = librosa.feature.rms(
        y=x,
        frame_length=1024,
        hop_length=256
    )[0]

    power = r**2

    limit = np.percentile(
        power,
        20
    )

    noise = power[
        power <= limit
    ]

    if len(noise) == 0:
        return np.nan

    noise_power = np.mean(noise)
    total_power = np.mean(power)

    speech_power = max(
        total_power - noise_power,
        EPS
    )

    return float(
        10 * np.log10(
            speech_power /
            (noise_power + EPS)
        )
    )

```

```

before = {
    "RMS (dB)": rms_db(audio),
    "Peak (dBFS)": peak_db(audio),
    "Noise Floor (dB)": noise_floor(audio),
    "Estimated SNR (dB)": estimated_snr(audio),
    "Speech Energy (dB)": speech_energy(audio),
    "High Frequency Energy (dB)": high_energy(audio)
}

after = {
    "RMS (dB)": rms_db(enhanced_audio),
    "Peak (dBFS)": peak_db(enhanced_audio),
    "Noise Floor (dB)": noise_floor(enhanced_audio),
    "Estimated SNR (dB)": estimated_snr(enhanced_audio),
    "Speech Energy (dB)": speech_energy(enhanced_audio),
    "High Frequency Energy (dB)": high_energy(enhanced_audio)
}

comparison = pd.DataFrame({
    "Parameter": list(before.keys()),
    "Before": list(before.values()),
    "After": list(after.values())
})

comparison["Change"] = (
    comparison["After"] -
    comparison["Before"]

```

```
)
display(comparison)
```

	Parameter	Before	After	Change
0	RMS (dB)	-31.388535	-21.918617	9.469917
1	Peak (dBFS)	-15.646233	-1.938200	13.708032
2	Noise Floor (dB)	-34.700546	-40.398186	-5.697639
3	Estimated SNR (dB)	3.148103	22.066025	18.917922
4	Speech Energy (dB)	-2.922974	7.612887	10.535861
5	High Frequency Energy (dB)	-15.930204	-15.216818	0.713387

```
f_before, p_before = welch(
    audio,
    fs=SR,
    nperseg=1024
)

f_after, p_after = welch(
    enhanced_audio,
    fs=SR,
    nperseg=1024
)

p_before_db = 10 * np.log10(
    p_before + EPS
)

p_after_db = 10 * np.log10(
    p_after + EPS
)

plt.figure(figsize=(13, 5))

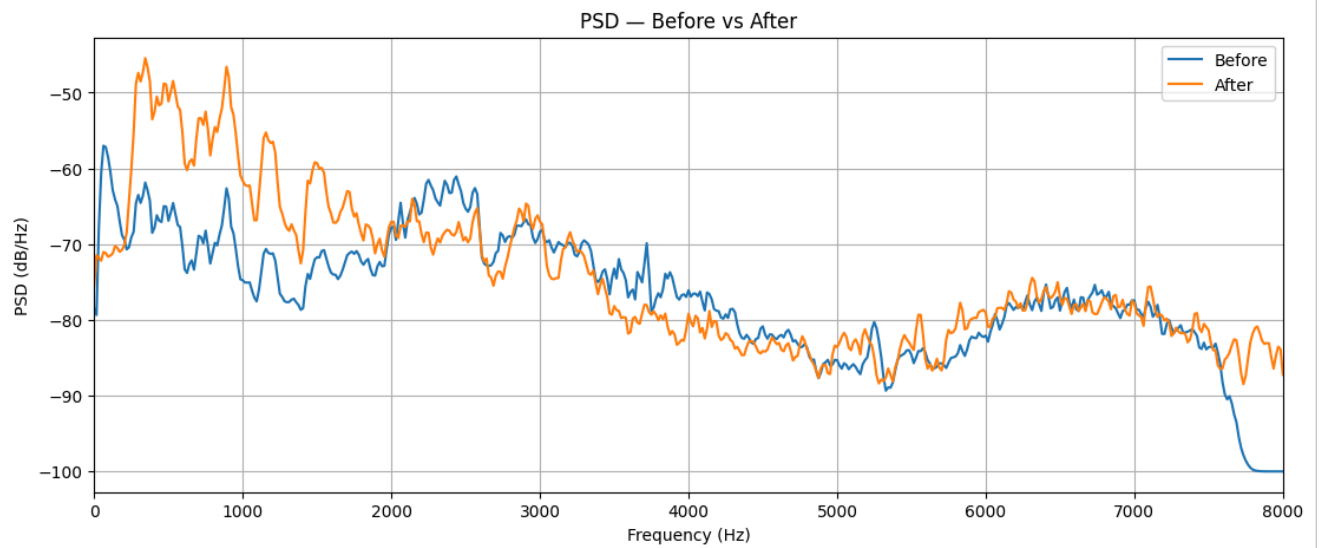
plt.plot(
    f_before,
    p_before_db,
    label="Before"
)

plt.plot(
    f_after,
    p_after_db,
    label="After"
)

plt.xlim(0, 8000)

plt.xlabel("Frequency (Hz)")
plt.ylabel("PSD (dB/Hz)")
plt.title("PSD - Before vs After")
plt.legend()
plt.grid(True)

plt.show()
```



```
t = np.arange(len(audio)) / SR

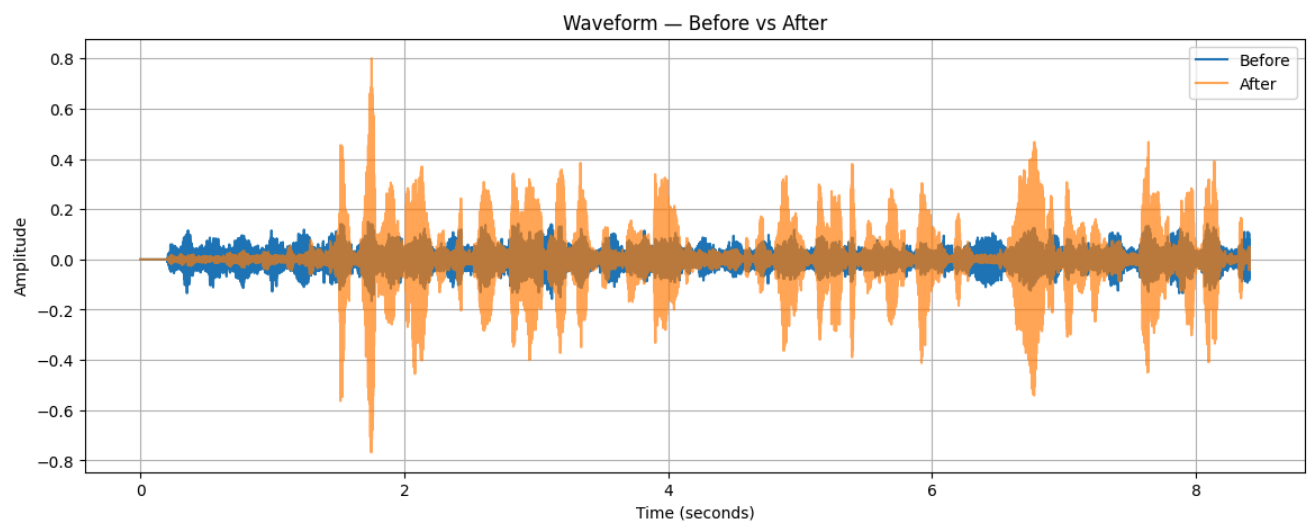
plt.figure(figsize=(14, 5))

plt.plot(
    t,
    audio,
    label="Before"
)

plt.plot(
    t,
    enhanced_audio,
    label="After",
    alpha=0.7
)

plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.title("Waveform — Before vs After")
plt.legend()
plt.grid(True)

plt.show()
```



```

D2 = librosa.stft(
    enhanced_audio,
    n_fft=1024,
    hop_length=256
)

S2 = librosa.amplitude_to_db(
    np.abs(D2),
    ref=np.max
)

plt.figure(figsize=(14, 5))

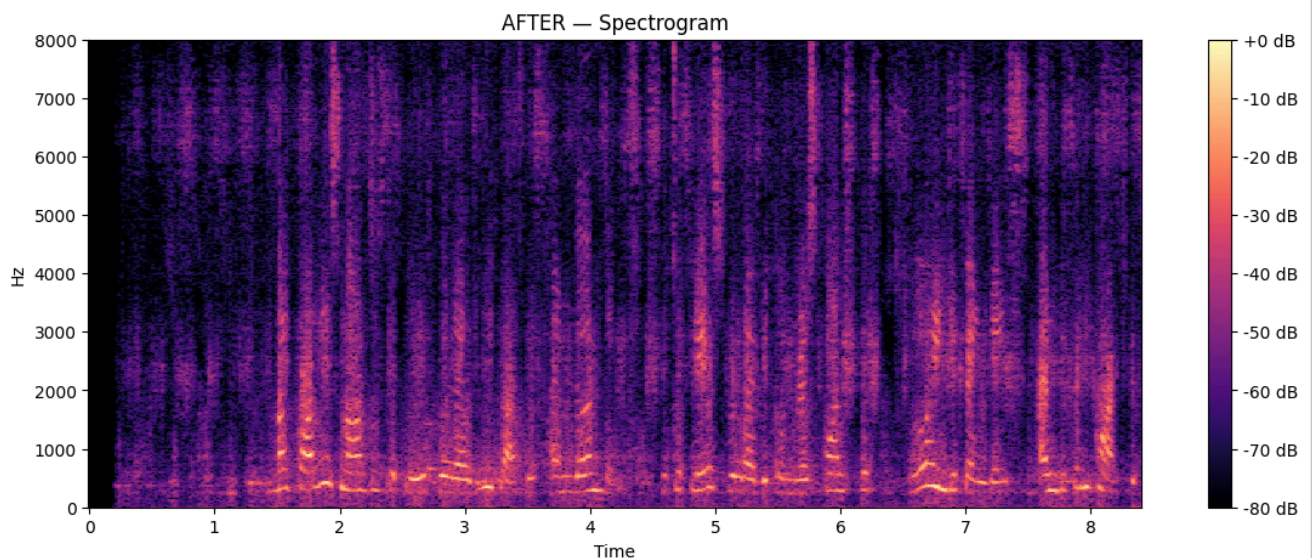
librosa.display.specshow(
    S2,
    sr=SR,
    hop_length=256,
    x_axis="time",
    y_axis="hz"
)

plt.colorbar(
    format="%+2.0f dB"
)

plt.title("AFTER - Spectrogram")
plt.ylim(0, 8000)

plt.show()

```



```

# =====
# CELL 19 - SPECTRAL DIFFERENCE
# =====

import numpy as np
import matplotlib.pyplot as plt
import librosa

# Make sure both signals have the same length
min_len = min(len(audio), len(enhanced_audio))

before = np.asarray(audio[:min_len], dtype=np.float32)
after = np.asarray(enhanced_audio[:min_len], dtype=np.float32)

# STFT
D_before = librosa.stft(
    before,
    n_fft=1024,
    hop_length=256,
    win_length=1024
)

```

```
D_after = librosa.stft(
    after,
    n_fft=1024,
    hop_length=256,
    win_length=1024
)

# Magnitude difference in dB
mag_before = np.abs(D_before)
mag_after = np.abs(D_after)

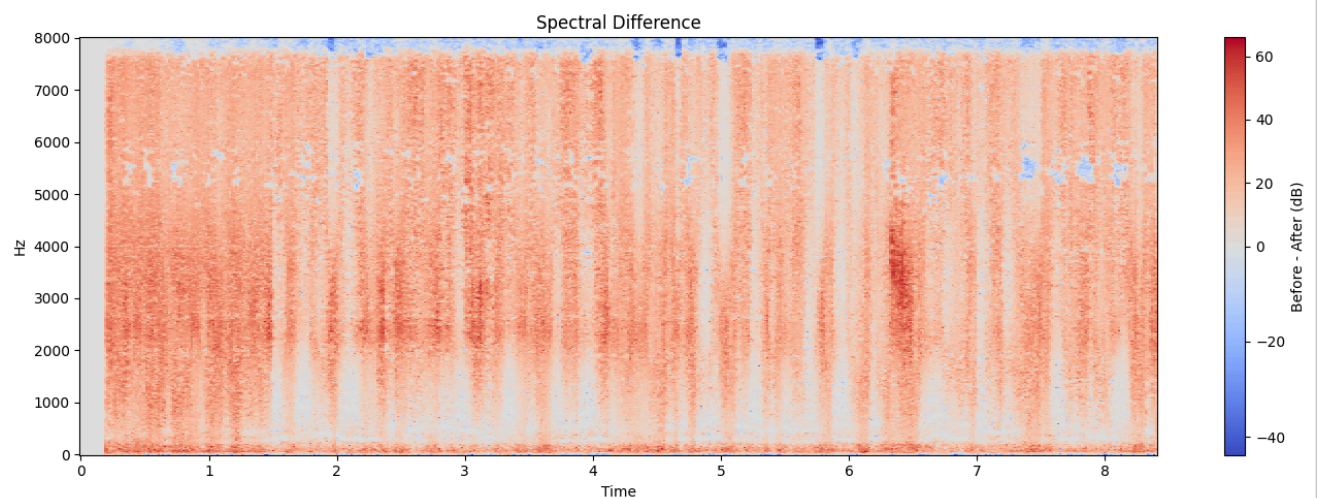
diff_db = librosa.amplitude_to_db(
    np.maximum(mag_before, 1e-8),
    ref=np.max
) - librosa.amplitude_to_db(
    np.maximum(mag_after, 1e-8),
    ref=np.max
)

plt.figure(figsize=(14, 5))

librosa.display.specshow(
    diff_db,
    sr=SR,
    hop_length=256,
    x_axis="time",
    y_axis="hz"
)

plt.colorbar(label="Before - After (dB)")
plt.title("Spectral Difference")
plt.tight_layout()
plt.show()

print("Spectral difference analysis completed.")
```



```

transcript = whisper_model.transcribe(
    OUTPUT_FILE,
    temperature=0,
    fp16=(DEVICE == "cuda")
)

print("Language:",
      transcript.get("language", "unknown"))

print()
print("TEXT:")
print(transcript["text"])

```

Language: en

TEXT:

My call is not just a place where I will be passed this time-learning. It's a place where I become responsible for independent

```

def srt_time(seconds):

    total_ms = int(
        round(seconds * 1000)
    )

    h = total_ms // 3600000
    total_ms %= 3600000

    m = total_ms // 60000
    total_ms %= 60000

    s = total_ms // 1000
    ms = total_ms % 1000

    return (
        f"{h:02d}:"
        f"{m:02d}:"
        f"{s:02d},"
        f"{ms:03d}"
    )

SRT_FILE = (
    "/content/enhanced_subtitles.srt"
)

with open(
    SRT_FILE,
    "w",
    encoding="utf-8"
) as f:

    number = 1

    for segment in transcript["segments"]:

        text = segment["text"].strip()

        if not text:
            continue

        f.write(
            f"{number}\n"
        )

        f.write(
            f"{srt_time(segment['start'])} --> "
            f"{srt_time(segment['end'])}\n"
        )

        f.write(
            f"{text}\n\n"
        )

        number += 1

print(

```

```

    "SRT created:",
    SRT_FILE
)

```

SRT created: /content/enhanced_subtitles.srt

```

# =====
# CELL 23 - SAVE REPORTS
# =====

import os
import numpy as np
import pandas as pd

# Create output directory
OUTPUT_DIR = "/content/audio_results"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# -----
# Before / After parameters
# -----

def calculate_parameters(x, sr):
    x = np.asarray(x, dtype=np.float32)

    peak = float(np.max(np.abs(x)))
    rms = float(np.sqrt(np.mean(x ** 2)) + 1e-12)

    rms_db = float(20 * np.log10(rms + 1e-12))
    peak_db = float(20 * np.log10(peak + 1e-12))

    duration = float(len(x) / sr)

    return {
        "Duration_seconds": duration,
        "RMS": rms,
        "RMS_db": rms_db,
        "Peak": peak,
        "Peak_db": peak_db
    }

before_params = calculate_parameters(audio, SR)
after_params = calculate_parameters(enhanced_audio, SR)

parameter_table = pd.DataFrame({
    "Parameter": list(before_params.keys()),
    "Before": list(before_params.values()),
    "After": list(after_params.values())
})

# Difference column
parameter_table["Difference"] = (
    parameter_table["After"] - parameter_table["Before"]
)

PARAM_FILE = os.path.join(
    OUTPUT_DIR,
    "before_after_parameters.csv"
)

parameter_table.to_csv(
    PARAM_FILE,
    index=False
)

print("Before/After Parameters:")
display(parameter_table)

print("\nSaved:")
print(PARAM_FILE)

# -----
# Save PSD data if available
# -----

```

```

if "freqs" in globals() and "noise_psd" in globals():

    psd_table = pd.DataFrame({
        "Frequency_Hz": np.asarray(freqs),
        "Noise_PSD": np.asarray(noise_psd)
    })

    PSD_FILE = os.path.join(
        OUTPUT_DIR,
        "noise_psd.csv"
    )

    psd_table.to_csv(
        PSD_FILE,
        index=False
    )

    print("Saved:")
    print(PSD_FILE)

else:
    print("PSD variables not available. Skipping PSD CSV.")

```

Before/After Parameters:

	Parameter	Before	After	Difference
0	Duration_seconds	8.405625	8.405625	0.000000
1	RMS	0.026951	0.080181	0.053230
2	RMS_dB	-31.388537	-21.918617	9.469919
3	Peak	0.165078	0.800000	0.634922
4	Peak_dB	-15.646233	-1.938200	13.708032

Saved:

/content/audio_results/before_after_parameters.csv
 PSD variables not available. Skipping PSD CSV.

```

# =====
# CELL 24 - CREATE FINAL ZIP
# =====

import os
import zipfile

OUTPUT_DIR = "/content/audio_results"

# Make sure directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Save enhanced audio again inside results folder
FINAL_AUDIO = os.path.join(
    OUTPUT_DIR,
    "enhanced_audio.wav"
)

sf.write(
    FINAL_AUDIO,
    enhanced_audio.astype(np.float32),
    SR,
    subtype="PCM_16"
)

print("Enhanced audio saved:")
print(FINAL_AUDIO)

# -----
# Add subtitle file if it exists
# -----

if "SRT_FILE" in globals() and os.path.exists(SRT_FILE):

    subtitle_destination = os.path.join(
        OUTPUT_DIR,
        "enhanced_subtitles.srt"
    )

```

```

    )

    # Copy subtitle file
    with open(SRT_FILE, "r", encoding="utf-8") as src:
        subtitle_text = src.read()

    with open(
        subtitle_destination,
        "w",
        encoding="utf-8"
    ) as dst:
        dst.write(subtitle_text)

    print("Subtitle saved:")
    print(subtitle_destination)

else:
    print("Subtitle file not found. Continuing without subtitles.")

# -----
# Create ZIP
# -----

ZIP_FILE = "/content/final_audio_results.zip"

# Remove old ZIP if it exists
if os.path.exists(ZIP_FILE):
    os.remove(ZIP_FILE)

with zipfile.ZipFile(
    ZIP_FILE,
    "w",
    compression=zipfile.ZIP_DEFLATED
) as zipf:

    for root, dirs, files in os.walk(OUTPUT_DIR):

        for file in files:

            full_path = os.path.join(root, file)

            # Store files without /content path
            arcname = os.path.relpath(
                full_path,
                OUTPUT_DIR
            )

            zipf.write(
                full_path,
                arcname
            )

print("\nZIP created successfully:")
print(ZIP_FILE)

# Show ZIP contents
with zipfile.ZipFile(ZIP_FILE, "r") as zipf:
    print("\nZIP contains:")
    for name in zipf.namelist():
        print(" - " + name)

```

```

Enhanced audio saved:
/content/audio_results/enhanced_audio.wav
Subtitle saved:
/content/audio_results/enhanced_subtitles.srt

```

```

ZIP created successfully:
/content/final_audio_results.zip

```

```

ZIP contains:
- enhanced_subtitles.srt
- enhanced_audio.wav
- before_after_parameters.csv

```

```

# =====
# CFI 25 - DOWNLOADED FINAL RESULTS

```